

In [1]: `import pandas as pd`

In [3]: `original_df=pd.read_csv("C:\\Users\\Mugdha\\Downloads\\predictive_maintenance.csv")`
`original_df.head()`

Out[3]:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Ty
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	Fail
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	Fail
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	Fail
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	Fail
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	Fail

In [14]: `real_df=original_df.drop('UDI',axis=1)`
`real_df=real_df.drop('Product ID',axis=1)`
`real_df.head()`

Out[14]:

	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	M	298.1	308.6	1551	42.8	0	0	No Failure
1	L	298.2	308.7	1408	46.3	3	0	No Failure
2	L	298.1	308.5	1498	49.4	5	0	No Failure
3	L	298.2	308.6	1433	39.5	7	0	No Failure
4	L	298.2	308.7	1408	40.0	9	0	No Failure

In [26]: `generated_data=pd.read_csv("E:\\MyMLproject\\syntab\\data\\New\\generated_data.csv")`
`generated_data.head()`

Out[26]:

	id	UDI	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	509	H	299.799988	308.600006	1419	48.799999	3	1	No Failure
1	2	7185	M	299.299988	310.000000	1550	37.044258	177	0	No Failure
2	3	9227	M	300.399994	310.902039	1583	35.799999	56	0	No Failure
3	4	1929	L	297.700012	307.799988	1646	28.988087	63	0	No Failure
4	5	4646	L	299.200012	309.000000	1244	66.368240	81	0	No Failure

In [27]: `synth_df=generated_data.drop('id',axis=1)`In [28]: `synth1_df=synth_df.drop('UDI',axis=1)`
`synth1_df.head()`

Out[28]:

	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	H	299.799988	308.600006	1419	48.799999	3	1	No Failure
1	M	299.299988	310.000000	1550	37.044258	177	0	No Failure
2	M	300.399994	310.902039	1583	35.799999	56	0	No Failure
3	L	297.700012	307.799988	1646	28.988087	63	0	No Failure
4	L	299.200012	309.000000	1244	66.368240	81	0	No Failure

In [38]: `from sklearn.preprocessing import LabelEncoder`
`le=LabelEncoder()`
`real_df['Type']=le.fit_transform(real_df['Type'])`
`synth1_df['Type']=le.fit_transform(synth1_df['Type'])`
`real_df['Failure Type']=le.fit_transform(real_df['Failure Type'])`
`synth1_df['Failure Type']=le.fit_transform(synth1_df['Failure Type'])`In [39]: `from sklearn.preprocessing import StandardScaler`
`sc=StandardScaler()`
`from sklearn.linear_model import LogisticRegression`
`x_train=synth1_df.drop('Failure Type',axis=1)`

```
x_train_scaled=sc.fit_transform(x_train)
y_train=synth1_df['Failure Type']
x_test=real_df.drop('Failure Type',axis=1)
x_test_scaled=sc.fit_transform(x_test)
y_test=real_df['Failure Type']
```

```
In [42]: from sklearn.metrics import classification_report
model=LogisticRegression(max_iter=1000)
model.fit(x_train_scaled,y_train)
y_pred=model.predict(x_test_scaled)
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.80	0.97	0.88	112
1	1.00	1.00	1.00	9652
2	0.65	0.92	0.77	78
3	0.90	0.74	0.81	95
4	0.00	0.00	0.00	18
5	1.00	0.38	0.55	45
accuracy			0.99	10000
macro avg	0.72	0.67	0.67	10000
weighted avg	0.99	0.99	0.99	10000

E:\anaconda\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

E:\anaconda\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

E:\anaconda\Lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

In []: